ARM PrimeCell Driver

UART v3.0 (PL010/PL011)

API Specification

PL010-ISPC-1000-C

Open Access

© Copyright ARM Limited 2000. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

Product status

The information in this document is Final (information on a developed product).

Web address

http://www.arm.com

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Document Summary

Document Issue	C
PrimeCell driver identifier	UART
Code Version	3.0
PrimeCell identifier	PL010/PL011

Contents

1 ABOUT THIS DOCUMENT

1.1 Change control

1.1.1 Change history

Issue	Date	Change
A		First issue of API
B	15/11/00	Revised document to match desired template formatFinal release
C	9/7/01	Updated to include DMA control functionality

2 SUMMARY

The PrimeCell UART peripheral is an AMBA slave module. The PrimeCell UART includes *an Infrared Data Association* (IrDA) *Serial InfraRed* (SIR) protocol *ENcoder/DECoder* (ENDEC).

The PrimeCell UART performs:

- serial-to-parallel conversion on data received from a peripheral device
- parallel-to-serial conversion on data transmitted to the peripheral device

3 FEATURES

The main features supported by the PrimeCell UART driver are:

- Programmable use of PrimeCell UART or IrDA/SIR input/output.
- Programmable FIFO disabling for 1-byte depth.
- Fully programmable serial interface characteristics:
  - data can be 5, 6, 7 or 8 bits
  - even, odd, stick or no-parity bit generation and detection
  - 1 or 2 stop bit generation
  - baud rate generation
- Data transmission, receipt, error and modem detection under interrupt control or manual polling.
- Programmable baud rate generator with support for up to 460.8Kbits/second, subject to reference clock frequency.
- Independent masking of transmit, receive and timeout, modem status and error condition interrupts.
- Support of the modem control functions CTS, DCD, DSR, RTS, DTR and RI.
- Programmable hardware flow control.
- Programmable FIFO levels for transmit and receive interrupt generation configuration.

4 DETAILS

- Peak performance obtained in tests: N/A
- ROM size (RO code + RO data) (release build): 4844 Bytes
- RAM size (ZI data) per instance (release build): 60 Bytes
- Does this driver support multiple PrimeCells?: YES
- Can these be detected at run-time? NO
- Does this driver support both endian-nesses?[1] YES
- Does this driver support Thumb?[2] YES

5 RELEASED FILES

`PL010-ISPC-1000-C`	This API document
`PL010/PL011-TREP-1000`	Test Results document
`apUART/apUART.h`	Public header file
`apUART/UART.h`	Private header file
`apUART/UART.c`	Source code
`apUART/UARTtst.c`	Self-test module

[1] To support both endian-nesses, the driver must be designed to be re-compiled in little-endian or big-endian form.

[2] The driver may include ARM code, but this must interwork with a Thumb build.

6 PUBLIC API

6.1 Type Definitions

6.1.1 apUART_eDCDEnable

Data Carrier Detect (DCD) state control setting. Used in apUART_DCDConfigGet and apUART_DCDConfigSet.

Declaration

```
typedef enum apUART_eDCDEnable
```

Elements

apUART_DCD_DISABLED	
apUART_DCD_ENABLED	DCD asserted.

6.1.2 apUART_eDTREnable

Data Transmit Ready (DTR) state control setting. Used in apUART_DTRConfigGet and apUART_DTRConfigSet.

Declaration

```
typedef enum apUART_eDTREnable
```

Elements

apUART_DTR_DISABLED	
apUART_DTR_ENABLED	DTR asserted.

6.1.3 apUART_eDMAMode

This specifies the DMA mode options. Applies to PL011 variant only.

Declaration

```
typedef enum apUARTSSP_eDMAMode
```

Elements

apUART_DMA_TX_ON	Transmit data using DMA enabled
apUART_DMA_TX_OFF	Transmit using DMA disabled
apUART_DMA_RX_ON	Receive data using DMA enabled
apUART_DMA_RX_OFF	Receive using DMA disabled

6.1.4 apUART_eError

General error reporting type for this module. Errors are returned typecast to the general error type apError.

Declaration

```
typedef enum apUART_eError
```

Elements

apUART_BUFFER_FULL	Receiving data buffer is full.
apUART_ERROR_BREAK	Break error (n/a to PL010).
apUART_ERROR_FRAME	Framing error (n/a to PL010).
apUART_ERROR_OVERRUN	Overrun error (n/a to PL010).
apUART_ERROR_PARITY	Parity error (n/a to PL010).
apUART_FIFO_EMPTY	Cannot read as Rx FIFO is empty
apUART_FIFO_FULL	Cannot write as Tx FIFO is full
apUART_IN_USE	UART is busy.
apUART_NO_BUFFER	No pBuffer specified for transmitting or receiving data.
apUART_SIZE_ZERO	Buffer of size zero.
apUART_INVALID_BAUD_RATE	Unable to generate divisor for required baud rate

6.1.5 apUART_eErrorEnable

(n/a for PL010). Enumerated type for use in defining a bitmask to mask Error Interrupts (apUART_ErrorIntSet) and comparing against the bitmask returned by apUART_ErrorIntGet to view the current Error Interrupt masking.

Declaration

```
typedef enum apUART_eErrorEnable
```

Elements

apUART_ERROR_INT_BREAK	Break error enable/disable.
apUART_ERROR_INT_FRAMING	Framing error enable/disable.
apUART_ERROR_INT_OVERRUN	Overrun error enable/disable.
apUART_ERROR_INT_PARITY	Parity error enable/disable.

6.1.6 apUART_eErrorFlags

Enumerated type for use in comparing against the value returned by apUART_Read for manual polling of errors (n/a for PL010).

Declaration

```
typedef enum apUART_eErrorFlags
```

Elements

apUART_ERROR_FLAG_BREAK	Break error detection.
apUART_ERROR_FLAG_FRAMING	Framing error detection.
apUART_ERROR_FLAG_OVERRUN	Overrun error detection.
apUART_ERROR_FLAG_PARITY	Parity error detection.

6.1.7 apUART_eErrorStatusFlags

Enumerated type for use in comparing against the bitmask apUART_ErrorStatusGet returns. (Primarily used for PL010).

Declaration

```
typedef enum apUART_eErrorStatusFlags
```

Elements

apUART_ERROR_STATUS_FLAG_BREAK	Break error detection.
apUART_ERROR_STATUS_FLAG_FRAMING	Framing error detection.
apUART_ERROR_STATUS_FLAG_OVERRUN	Overrun error detection.
apUART_ERROR_STATUS_FLAG_PARITY	Parity error detection.

6.1.8 apUART_eFIFOLevelSelect

(n/a for PL010). FIFO level select. Sets trigger levels for Transmit and Receive interrupts. Used in `apUART_sConfigFIFOs` type.

Declaration

```
typedef enum apUART_eFIFOLevelSelect
```

Elements

apUART_HALF	receive FIFO >= 1/2 full (<= for transmit).
apUART_ONEEIGHTH	receive FIFO >= 1/8 full (<= 1/8 for transmit).
apUART_ONEQUARTER	receive FIFO >= 1/4 full (<= for transmit).
apUART_SEVENEIGHTHS	receive FIFO >= 7/8 full (<= 7/8 for transmit).
apUART_THREEQUARTERS	receive FIFO >= 3/4 full (<= for transmit).

6.1.9 apUART_eHWFlowSelect

(n/a for PL010). Hardware flow control enumerated type. Used in `apUART_sConfigData` type.

Declaration

```
typedef enum apUART_eHWFlowSelect
```

Elements

apUART_HWFLOW_ALL	Hardware flow control fully enabled.
apUART_HWFLOW_CTS_ONLY	Hardware flow control of outgoing data.

apUART_HWFLOW_NONE	Hardware flow control disabled.
apUART_HWFLOW_RTS_ONLY	Hardware flow control of incoming data.

6.1.10 apUART_eListenerMessage

UART port activity information, which is passed from the Interrupt Service Routine to the user-defined `apUART_rListener` function.

Declaration

```
typedef enum apUART_eListenerMessage
```

Elements

apUART_DATA_ARRIVED	Incoming data without an apUART_Receive action setup to receive the data.
apUART_DEVICE_CHANGED	A modem signal has changed (PL010 only).
apUART_DEVICE_CHANGED_CTS	Modem has signalled CTS change in state (n/a for PL010).
apUART_DEVICE_CHANGED_DCD	Modem has signalled DCD change in state (n/a for PL010).
apUART_DEVICE_CHANGED_DSR	Modem has signalled DSR change in state (n/a for PL010).
apUART_DEVICE_CHANGED_RI	Modem has signalled RI change in state (n/a for PL010).

6.1.11 apUART_eLoopBackEnable

Loop back enable enumerated type. Used in `apUART_sConfigIR` type.

Declaration

```
typedef enum apUART_eLoopBackEnable
```

Elements

apUART_LOOPBACK_DISABLED	Loop back disabled.
apUART_LOOPBACK_ENABLED	Loop back enabled.

6.1.12 apUART_eLowPowerMode

Low power mode enumerated type. When set to low power mode, low-level bits are transmitted with a pulse width 3 times the period of the low power baud setting, LowPowerBaud16, regardless of the selected bit rate. This setting uses less power, but might reduce the transmission distance. Used in `apUART_sConfigPower` type.

Declaration

```
typedef enum apUART_eLowPowerMode
```

Elements

apUART_MODE_LOW_POWER	Low power mode.
apUART_MODE_NORMAL	Normal power mode.

6.1.13 apUART_eModemEnable

(n/a for PL010). Enumerated type for use in defining and comparing against a bitmask to set and retrieve the Modem interrupt mask. Used in apUART_ModemIntSet and apUART_ModemIntGet.

Declaration

```
typedef enum apUART_eModemEnable
```

Elements

apUART_MODEM_INT_CTS	Enable/disable CTS interrupt if supported (n/a for PL010).
apUART_MODEM_INT_DCD	Enable/disable DCD interrupt if supported (n/a for PL010).
apUART_MODEM_INT_DISABLE	Disable all the Modem interrupts.
apUART_MODEM_INT_DSR	Enable/disable DSR interrupt if supported (n/a for PL010).
apUART_MODEM_INT_ENABLE	Enable all the Modem interrupts.
apUART_MODEM_INT_RI	Enable/disable RI interrupt if supported (n/a for PL010).

6.1.14 apUART_eModemFlags

Enumerated type for use in comparing against the bitmask apUART_ModemStatusGet returns.

Declaration

```
typedef enum apUART_eModemFlags
```

Elements

apUART_MODEM_FLAG_CTS	CTS asserted.
apUART_MODEM_FLAG_DCD	DCD asserted.
apUART_MODEM_FLAG_DSR	DSR asserted.
apUART_MODEM_FLAG_RI	RI asserted (n/a for PL010).

6.1.15 apUART_eParitySelect

Parity select enumerated type. Used with `apUART_sConfigData` type.

Declaration

```
typedef enum apUART_eParitySelect
```

Elements

apUART_EVEN_PARITY	Even parity checking.
apUART_NO_PARITY	Parity checking disabled.
apUART_ODD_PARITY	Odd parity checking.

6.1.16 apUART_eRIEnable

(n/a for PL010). Ring Indicator (RI) setting state control. Used in apUART_RIConfigSet and apUART_RIConfigGet.

Declaration

```
typedef enum apUART_eRIEnable
```

Elements

apUART_RI_DISABLED	
apUART_RI_ENABLED	RI asserted.

6.1.17 apUART_eRTSEnable

(n/a for PL010). Request To Send (RTS) setting state control. Used in apUART_RTSConfigSet and apUART_RTSConfigGet.

Declaration

```
typedef enum apUART_eRTSEnable
```

Elements

apUART_RTS_DISABLED	
apUART_RTS_ENABLED	RTS asserted.

6.1.18 apUART_eSIREnable

SIR enable. When enabled, data is transmitted and received on nSIROUT and SIRIN. When disabled, nSIROUT remains clear, and signal transitions on SIRIN have no effect. Used in `apUART_sConfigIR` type.

Declaration

```
typedef enum apUART_eSIREnable
```

Elements

apUART_SIR_DISABLED	SIR disabled.
apUART_SIR_ENABLED	SIR enabled.

6.1.19 apUART_eStickParityEnable

(n/a for PL010). Stick parity select enumerated type. Used in `apUART_sConfigData` type.

Declaration

```
typedef enum apUART_eStickParityEnable
```

Elements

apUART_STICK_DISABLED	Stick parity disabled.
apUART_STICK_ENABLED	Stick parity enabled.

6.1.20 apUART_eStopBits

Stop bits enumerated type. Used in `apUART_sConfigData` type.

Declaration

```
typedef enum apUART_eStopBits
```

Elements

apUART_ONE_STOP_BIT	One stop bit.
apUART_TWO_STOP_BITS	Two stop bits.

6.1.21 apUART_eWordLength

Word length enumerated type. Used in `apUART_sConfigData` type.

Declaration

```
typedef enum apUART_eWordLength
```

Elements

apUART_BITS_5	5 bits.
apUART_BITS_6	6 bits.
apUART_BITS_7	7 bits.
apUART_BITS_8	8 bits.

6.1.22 apUART_rCallback

Callback for apUART_Transmit and apUART_Receive indicating completion of the action or an error. The interrupt service routine calls this function with a message regarding the current transmit or receive action setup by apUART_Transmit and apUART_Receive.

Declaration

```
typedef void (*apUART_rCallback) (UWORD32 Id, apError ErrorMessage)
```

Remarks

Parameters are the UART identifier, Id, and the apUART_eError code, ErrorMessage, pertaining to the last transfer. ErrorMessage should be checked against:

- apERR_NONE if successful - for apUART_Receive callbacks it is recommended that the user calls apUART_ReceiveStatusGet to see what length of pBuffer contains data. For apUART_Transmit this message indicates the transmission has completed. If transmission hasn't ended another apUART_Receive call should be intiated or apUART_Continue or apUART_TerminateReceive.
- apUART_BUFFER_FULL if pBuffer has been filled up (apUART_Receive only).
- apUART_ERROR_FRAME if there has been a framing error (apUART_Receive only).
- apUART_ERROR_PARITY if there has been a parity error (apUART_Receive only).
- apUART_ERROR_BREAK if there has been a break error (apUART_Receive only).
- apUART_ERROR_OVERRUN if there has been an overrun error (apUART_Receive only).

6.1.23 apUART_rListener

Callback for apUART_CallbackSet. This function is called under interrupts if a receive action is not underway and data arrives (ie. no `apUART_rCallback` function registered), or if there is a change in modem status.

Declaration

```
typedef void (*apUART_rListener) (UWORD32 Id, apUART_eListenerMessage eMessage)
```

Remarks

Parameters are the UART identifier, Id, and apUART_eListenerMessage information, eMessage, pertaining to the change in UART status. eMessage should be checked against:

- apUART_DATA_ARRIVED - if data has arrived and the UART is not already setup to read the data.
- apUART_DEVICE_CHANGED - if a modem status interrupt has occurred (PL010 only).
- apUART_DEVICE_CHANGED_RI - if a RI status interrupt has ocurred (n/a for PL010).
- apUART_DEVICE_CHANGED_CTS - if a CTS status interrupt has ocurred (n/a for PL010).
- apUART_DEVICE_CHANGED_DCD - if a DCD status interrupt has ocurred (n/a for PL010).
- apUART_DEVICE_CHANGED_DSR - if a DSR status interrupt has ocurred (n/a for PL010).

6.1.24 apUART_sConfigData

UART data transfer configuration data. Used with apUART_DataConfigSet and apUART_DataConfigGet.

Declaration

```
struct apUART_sConfigData
{
    apUART_eWordLength              eWordLen;
    apUART_eStopBits                eStopBits;
    apUART_eParitySelect            eParitySelect;
    apUART_eStickParityEnable       eStickParity;
    apUART_eHWFlowSelect            eHWFlowSelect;
}
```

Elements

eHWFlowSelect	Hardware flow enable/disable control where supported (ignored for PL010).
eParitySelect	Parity select (odd, even or disabled).
eStickParity	Stick parity enable/disable where supported (ignored for PL010).
eStopBits	Number of stop bits to send (one or two).
eWordLen	Word length (5, 6, 7 or 8 bits).

Remarks

This structure is used with apUART_DataConfigSet and apUART_DataConfigGet to configure the UART's line control parameters.

6.1.25 apUART_sConfigFIFOs

(n/a for PL010). UART FIFO configuration data. Used with apUART_FIFOConfigSet and apUART_FIFOConfigGet.

Declaration

```
struct apUART_sConfigFIFOs
{
    apUART_eFIFOLevelSelect         eReceiveFIFOLevel;
    apUART_eFIFOLevelSelect         eTransmitFIFOLevel;
}
```

Elements

eReceiveFIFOLevel	Receive FIFO interrupt level setting.
eTransmitFIFOLevel	Transmit FIFO interrupt level setting.

Remarks

This structure is used with apUART_FIFOConfigSet and apUART_FIFOConfigGet to configure the UART's FIFO levels for transmit and receive interrupt triggering.

6.1.26 apUART_sConfigIR

UART IR configuration data. Used with apUART_IRConfigSet and apUART_IRConfigGet.

Declaration

```
struct apUART_sConfigIR
{
    apUART_eLoopBackEnable          eLoopBackEnable;
    apUART_eSIREnable               eSIREnable;
}
```

Elements

eLoopBackEnable	Loop back enable.
eSIREnable	SIR enable.

Remarks

This structure is used with apUART_IRConfigSet and apUART_IRConfigGet to configure the UART's IrDA SIR capabilities.

6.1.27 apUART_sConfigPower

UART low power configuration data. Used with apUART_PowerConfigSet and apUART_PowerConfigGet.

Declaration

```
struct apUART_sConfigPower
{
    UWORD32                     LowPowerBaud16;
    apUART_eLowPowerMode        eLowPowerMode;
}
```

Elements

LowPowerBaud16	Low power baud rate.
eLowPowerMode	Low power mode.

Remarks

This structure is used with apUART_PowerConfigSet and apUART_PowerConfigGet to configure the UART's IrDA power saving features.

6.1.28 apUART_sInitialData

UART initialization data. Used with apUART_Initialize.

Declaration

```
struct apUART_sInitialData
{
  UWORD32 FIFOSize;
  UWORD32 ClockFrequency;
}
```

Elements

FIFOSize	Size of Rx FIFO in bytes.
ClockFrequency	Reference clock frequency applied to UARTCLK. Informs the timer driver functions of the external clock frequency used to drive the UART. This is required for the apUART_BaudRateSet, apUART_BaudRateGet, apUART_PowerConfigSet and apUART_PowerConfigGet functions to allow them to convert from seconds and Hz to and from clock ticks.

Remarks

These are device parameters, which are either defined by the system hardware or not intended to be altered except through rebuilding the system.

6.2 External Functions

6.2.1 apUART_BaudRateGet

Returns the Baud rate that the UART is operating at.

Declaration

```
PUBLIC UWORD32 apUART_BaudRateGet(apOS_UART_oId Id)
```

Parameters

Id	UART Id.

Remarks

Complement of apUART_BaudRateSet. Reads the Baud rate divisor and uses a local copy of the clock frequency to calculate the UART Baud rate (clock frequency set by apUART_Initialize).

6.2.2 apUART_BaudRateSet

Configures the UART to a specific Baud rate.

Declaration

```
PUBLIC apError void apUART_BaudRateSet(apOS_UART_oId Id, UWORD32 Rate)
```

Parameters

Id	UART Id.
Rate	Baud rate in bps.

Return Value

- apERR_NONE if successful.
- apUART_INVALID_BAUD_RATE if unable to set divisor for required BAUD rate.

Remarks

Sets the Baud rate divisor based on a local copy of the clock frequency (set by apUART_Initialize).

6.2.3 apUART_CallbackSet

Sets the user rListener function for UART Rx port activity and Modem interrupts.

Declaration

```
PUBLIC void apUART_CallbackSet(apOS_UART_oId Id, apUART_rListener rListener)
```

Parameters

Id	UART Id.
rListener	The callback function of type `apUART_rListener`; setting it to aRNULL will clear the callback.

Remarks

If anything happens at the Rx port and a transfer is not underway, or a Modem interrupt occurs, the rListener function is invoked and provided with the appropriate apUART_eListenerMessage parameter.

6.2.4 apUART_Continue

Used to resume receiving data under interrupts if the port times out before the required message length has been received (ie. rCallback routine informed of apERR_NONE but apUART_ReceiveStatusGet informs the user that there is still more than the expected amount of buffer space left).

Declaration

```
PUBLIC apError apUART_Continue(apOS_UART_oId Id)
```

Parameters

Id	UART Id.

Return Value

- apERR_NONE if successful.
- apUART_IN_USE if the UART is already receiving data.

Remarks

Resumes previous transfer.

6.2.5 apUART_DCDConfigGet

Returns the Data Carrier Detect configuration (DCE role). (n/a for PL010 and Integrator variants)

Declaration

```
PUBLIC apError apUART_DCDConfigGet(apOS_UART_oId Id,
                                   apUART_eDCDEnable *pConfig)
```

Parameters

Id	UART Id.
pConfig	pointer to apUART_eDCDEnable type is returned indicating the DCD configuration

Return Value

- apERR_NONE if successful.
- apERR_UNSUPPORTED if unsupported by hardware (PL010).

Remarks

Returns the status of the DCD output. Reads the relevant Out1 bit in the Control register.

6.2.6 apUART_DCDConfigSet

Configures the Data Carrier Detect output (DCE role). (n/a for PL010 and Integrator variants)

Declaration

```
PUBLIC apError apUART_DCDConfigSet(apOS_UART_oId Id, apUART_eDCDEnable Config)
```

Parameters

Id	UART Id.
Config	apUART_eDCDEnable type

Return Value

- apERR_NONE if successful.
- apERR_UNSUPPORTED if unsupported by hardware (PL010).

Remarks

Asserts or de-asserts the DCD output. Sets the relevant Out1 bit in the Control register.

6.2.7 apUART_DMAAddressGet

Gets the address of the data register for use by the DMA controller. Applies to PL011 variant only.

Declaration

```
PUBLIC UWORD32 apUART_DMAAddressGet(apOS_UART_oId oId)
```

Parameters

oId	UART device identifier

Return Value

Data Register address

6.2.8 apUART_DMAModeSet

Sets or clears the DMA enable bit in the DMA control register. Applies to PL011 variant only.

Declaration

```
PUBLIC void apUART_DMAModeSet(apOS_UART_oId oId,
                              apUART_eDMAMode DMAMode)
```

Parameters

oId	UART device identifier
DMAMode	Sets DMA mode on or off

Return Value

None

Remarks

Enables the DMA logic in the UART for receive or transmit of data.

6.2.9 apUART_DTRConfigGet

Returns the Data Transmit Ready configuration. (n/a for PL010 and Integrator variants)

Declaration

```
PUBLIC apError apUART_DTRConfigGet(apOS_UART_oId Id,
                                   apUART_eDTREnable *pConfig)
```

Parameters

Id	UART Id.
pConfig	pointer to apUART_eDTREnable type is returned indicating the DTR configuration.

Return Value

- apERR_NONE if successful.
- apERR_UNSUPPORTED if unsupported by hardware (PL010).

Remarks

Returns the status of the DTR output. Reads the relevant DTR bit in the Control register.

6.2.10 apUART_DTRConfigSet

Configures the Data Transmit Ready output. (n/a for PL010 and Integrator variants)

Declaration

```
PUBLIC apError apUART_DTRConfigSet(apOS_UART_oId Id, apUART_eDTREnable Config)
```

Parameters

Id	UART Id
Config	apUART_eDTREnable type.

Return Value

- apERR_NONE if successful.
- apERR_UNSUPPORTED if unsupported by hardware (PL010).

Remarks

Asserts or deasserts the DTR output. Sets the relevant DTR bit in the Control register.

6.2.11 apUART_DataConfigGet

Returns the data transfer options currently set.

Declaration

```
PUBLIC void apUART_DataConfigGet(apOS_UART_oId Id, apUART_sConfigData *pConfig)
```

Parameters

Id	UART Id.
pConfig	pointer to `apUART_sConfigData` struct

Remarks

The complement of apUART_DataConfigSet(). Values returned are the character frame parameters, data bits, stop bits and parity. Hardware flow control and stick parity configuration is only valid where supported.

6.2.12 apUART_DataConfigSet

Configure data transfer options.

Declaration

```
PUBLIC void apUART_DataConfigSet(apOS_UART_oId Id, apUART_sConfigData *pConfig)
```

Parameters

Id	UART Id.
pConfig	pointer to `apUART_sConfigData` struct

Remarks

Write values to hardware; values set are the character frame parameters, data bits, stop bits and parity. Hardware flow control and stick parity is configured where supported.

6.2.13 apUART_Disable

Disables the UART.

Declaration

```
PUBLIC void apUART_Disable(apOS_UART_oId Id)
```

Parameters

Id	Selects the UART to be disabled.

Remarks

The specified UART is disabled. This call should be invoked before any configuration changes

6.2.14 apUART_Enable

Enables the UART.

Declaration

```
PUBLIC void apUART_Enable(apOS_UART_oId Id)
```

Parameters

Id	Selects the UART to be enabled.

Remarks

The specified UART's FIFOs, all the interrupts excluding transmit and the UART itself are enabled.

6.2.15 apUART_ErrorIntGet

Returns the status of the Error interrupts. (n/a for PL010 and Integrator variants)

Declaration

```
PUBLIC apError apUART_ErrorIntGet(apOS_UART_oId Id, UWORD32 *pErrormask)
```

Parameters

Id	UART Id.
pErrormask	Pointer to a bitmask the function uses to return the Error interrupt status. This value returned should be ANDed with the apUART_eErrorEnable type values to determine the Error interrupt status.

Return Value

- apERR_NONE if successful.
- apERR_UNSUPPORTED if unsupported by hardware (PL010).

Remarks

Returns the value of the relevant interrupt mask bits.

6.2.16 apUART_ErrorIntSet

Enables/disables the specified Error interrupts. (n/a for PL010 and Integrator variants)

Declaration

```
PUBLIC apError apUART_ErrorIntSet(apOS_UART_oId Id, UWORD32 Errormask)
```

Parameters

Id	UART Id.
Errormask	A bitmask created by ORing values of the type apUART_eErrorEnable together

for those Error interrupts to enable.

Return Value

- apERR_NONE if successful.
- apERR_UNSUPPORTED if unsupported by hardware (PL010).

Remarks

Sets the relevant interrupt mask bits.

6.2.17 apUART_ErrorStatusClear

Clears the current status information for the data character read prior to the call to this function.

Declaration

```
PUBLIC void apUART_ErrorStatusClear(apOS_UART_oId Id)
```

Parameters

Id	UART Id.

Remarks

Writes to the receive status/error clear register to clear the status.

6.2.18 apUART_ErrorStatusGet

Returns the status information for the byte read prior to the call to this function. This function call is included for backward compatibility with PL010. PL011 variants should use the method outlined in apUART_Read for the polling method of error detection. 1 pCLK cycle is needed inbetween a call to apUART_Read and apUART_ErrorStatusGet to allow the receive status/error clear register to update.

Declaration

```
PUBLIC UWORD32 apUART_ErrorStatusGet(apOS_UART_oId Id)
```

Parameters

Id	UART Id.

Return Value

A value that should be AND'ed with the values in apUART_eErrorStatusFlags to determine the current error status.

Remarks

Returns the value of the receive status/error clear register.

6.2.19 apUART_FIFOConfigGet

Returns FIFO watermark configuration. (n/a for PL010 and Integrator variants)

Declaration

```
PUBLIC apError apUART_FIFOConfigGet(apOS_UART_oId Id,
                                    apUART_sConfigFIFOs *pConfig)
```

Parameters

Id	UART Id.
pConfig	pointer to `apUART_sConfigFIFOs` struct

Return Value

- apERR_NONE if successful.
- apERR_UNSUPPORTED if unsupported by hardware (PL010).

Remarks

Complement of apUART_FIFOConfigSet(). Values returned are the transmit and receive FIFO levels.

6.2.20 apUART_FIFOConfigSet

Configures FIFO watermark levels. (n/a for PL010 and Integrator variants)

Declaration

```
PUBLIC apError apUART_FIFOConfigSet(apOS_UART_oId Id,
                                    apUART_sConfigFIFOs *pConfig)
```

Parameters

Id	UART Id.
pConfig	pointer to `apUART_sConfigFIFOs` struct

Return Value

- apERR_NONE if successful.
- apERR_UNSUPPORTED if unsupported by hardware (PL010).

Remarks

Write values to hardware. Values set are the transmit and receive FIFO levels.

6.2.21 apUART_FIFODisable

Disables FIFOs for a given UART.

Declaration

```
PUBLIC void apUART_FIFODisable(apOS_UART_oId Id)
```

Parameters

Id	selects the UART to be referenced.

Remarks

The specified UART's FIFOs are disabled by setting the relevant bit in the LineCon_H register.

6.2.22 apUART_FIFOEnable

Enables FIFOs for given UART.

Declaration

```
PUBLIC void apUART_FIFOEnable(apOS_UART_oId Id)
```

Parameters

Id	selects the UART to be referenced.

Remarks

The specified UART FIFOs are enabled by setting the relevant bit in the LineCon_H register.

6.2.23 apUART_IRConfigGet

Returns the configuration of the UART's infra-red and loopback options.

Declaration

```
PUBLIC void apUART_IRConfigGet(apOS_UART_oId Id, apUART_sConfigIR *pConfig)
```

Parameters

Id	UART Id.
pConfig	pointer to power `apUART_sConfigIR` struct

Remarks

The complement of apUART_IRConfigSet(). Values returned are the loopback enable and the infra-red enable. When using an Integrator UART the infra-red configuration can be ignored as it is not available.

6.2.24 apUART_IRConfigSet

Configures the UART's infra-red and loopback settings.

Declaration

```
PUBLIC void apUART_IRConfigSet(apOS_UART_oId Id, apUART_sConfigIR *pConfig)
```

Parameters

Id	UART Id.
pConfig	pointer to IR `apUART_sConfigIR` struct

Remarks

Values set are the loopback enable and the infra-red enable. Before enabling the SIR, apUART_Disable should be called. When using an Integrator UART the infra-red apUART_eSIREnable should always be defined as disabled (apUART_SIR_DISABLED).

6.2.25 apUART_Initialize

Initializes the UART.

Declaration

```
PUBLIC void apUART_Initialize (apOS_UART_oId Id,
                               apOS_System_eBaseAddress eBase,
                               UWORD32 Interrupts,
                               CONST apOS_INT_oInterruptSource * pSources,
                               CONST apUART_sInitialData *pInitial)
```

Parameters

Id	UART to initialize.
eBase	base address of UART registers.
Interrupts	Number of interrupt events recognised by handler.
pSources	Pointer to the array of interrupt events handled.
pInitial	Pointer to the initialization data, containing the FIFO size.

Remarks

The initialization routine carries out the following:

- Stores the UART's base address and interrupt Id.
- Removes any registered callbacks.
- Disables UART, SIR, IrDA low power and loop back.
- Sets word length to 8, enables the FIFOs to 1 byte registers, one stop bit, no parity and send break = 0.
- Empties any data in the receive FIFO.
- Where supported - enables transmit and receive sections, de-asserts DTR, RTS, DCD (Out1) and RI(Out2), disables stick parity and HW flow control.

6.2.26 apUART_ModemIntGet

Returns the settings of the Uart's Modem interrupt mask.

Declaration

```
PUBLIC void apUART_ModemIntGet(apOS_UART_oId Id, UWORD32 *pModemmask)
```

Parameters

Id	UART Id.
pModemmask	Pointer to a bitmask the function uses to return the Modem interrupt mask. This value should be ANDed with apUART_eModemEnable type values to determine the Modem interrupt status.

Remarks

Returns the value of the relevant interrupt mask bits.

6.2.27 apUART_ModemIntSet

Enables/disables the Uart's specified Modem interrupts.

Declaration

```
PUBLIC apError apUART_ModemIntSet(apOS_UART_oId Id, UWORD32 Modemmask)
```

Parameters

Id	UART Id.
Modemmask	a bitmask defined by ORing values of the type apUART_eModemEnable together for the Modem interrupts to enable. Passing 0 disables all the interrupts.

Return Value

- apERR_NONE if successful.
- apERR_UNSUPPORTED if unsupported by hardware.

Remarks

Sets the relevant interrupt mask bit(s).

6.2.28 apUART_ModemStatusGet

Primarily to be used within the `apUART_rListener` routine. Determines the status of the incoming modem signals CTS, DSR, DCD and RI (RI where supported).

Declaration

```
PUBLIC UWORD32 apUART_ModemStatusGet(apOS_UART_oId Id)
```

Parameters

Id	UART Id.

Return Value

A value that should be AND'ed with values from the type apUART_eModemFlags to determine the state of the incoming modem signals.

Remarks

Returns the values of the modem flags.

6.2.29 apUART_PowerConfigGet

Returns the UART's low power settings. (n/a to Integrator UARTs)

Declaration

```
PUBLIC apError apUART_PowerConfigGet(apOS_UART_oId Id,
                                     apUART_sConfigPower *pConfig)
```

Parameters

Id	UART Id.
pConfig	pointer to `apUART_sConfigPower` struct

Return Value

- apERR_NONE if successful.
- apERR_UNSUPPORTED if unsupported by hardware .

Remarks

The complement of apUART_PowerConfigSet(). Values returned are the low power mode and the low power baud rate, calculated from the low power counter divisor and a local copy of the clock frequency (set by apUART_ClockFrequencyNotify).

6.2.30 apUART_PowerConfigSet

Configures the UART's low power options. (n/a to Integrator UARTs)

Declaration

```
PUBLIC apError void apUART_PowerConfigSet(apOS_UART_oId Id,
                                     apUART_sConfigPower *pConfig)
```

Parameters

Id	UART Id.
pConfig	pointer to `apUART_sConfigPower` struct

Return Value

- apERR_NONE if successful.
- apERR_UNSUPPORTED if unsupported by hardware.

Remarks

Write values to hardware. Values set are the low power mode and the low power counter divisor based on a local copy of the clock frequency (set by apUART_ClockFrequencyNotify).

6.2.31 apUART_RIConfigGet

Returns the Ring Indicator configuration (DCE role). (n/a for PL010 and Integrator variants)

Declaration

```
PUBLIC apError apUART_RIConfigGet(apOS_UART_oId Id, apUART_eRIEnable *pConfig)
```

Parameters

Id	UART Id.
pConfig	pointer to apUART_eRIEnable type is returned indicating the RI configuration

Return Value

- apERR_NONE if successful.
- apERR_UNSUPPORTED if unsupported by hardware (PL010).

Remarks

Returns the status of the RI output. Reads the Out2 bit in the Control register.

6.2.32 apUART_RIConfigSet

Configures the Ring Indicator output (DCE role). (n/a for PL010 and Integrator variants)

Declaration

```
PUBLIC apError apUART_RIConfigSet(apOS_UART_oId Id, apUART_eRIEnable Config)
```

Parameters

Id	UART Id.
Config	apUART_eRIEnable type.

Return Value

- apERR_NONE if successful.
- apERR_UNSUPPORTED if unsupported by hardware (PL010).

Remarks

Asserts or de-asserts the RI output. Sets the Out2 bit in the Control register.

6.2.33 apUART_RTSConfigGet

Returns the Request To Send configuration. (n/a for PL010 and Integrator variants)

Declaration

```
PUBLIC apError apUART_RTSConfigGet(apOS_UART_oId Id,
                                   apUART_eRTSEnable *pConfig)
```

Parameters

Id	UART Id.
pConfig	pointer to apUART_eRTSEnable type is returned indicating the RTS configuration

Return Value

- apERR_NONE if successful.
- apERR_UNSUPPORTED if unsupported by hardware (PL010).

Remarks

Returns the status of the RTS output. Reads the RTS bit in the Control register.

6.2.34 apUART_RTSConfigSet

Configures the Request To Send output. (n/a for PL010 and Integrator variants)

Declaration

```
PUBLIC apError apUART_RTSConfigSet(apOS_UART_oId Id, apUART_eRTSEnable Config)
```

Parameters

Id	UART Id.
Config	apUART_eRTSEnable type.

Return Value

- apERR_NONE if successful.
- apERR_UNSUPPORTED if unsupported by hardware (PL010).

Remarks

Asserts or de-asserts the RTS output. Sets the RTS bit in the Control register. The RTS output cannot be controlled if RTS hardware flow control is supported and enabled. Data is only requested when there is space in the receive FIFO for it to be received.

6.2.35 apUART_Read

Read a byte of data and any error conditions from the UART (error conditions n/a for PL010).

Declaration

```
PUBLIC UWORD32 apUART_Read(apOS_UART_oId Id)
```

Parameters

Id	UART Id.

Return Value

- Data value and error conditions read from UART (errors n/a for PL010).
- apUART_FIFO_EMPTY if there is no data waiting.
- apUART_IN_USE if the UART is already in use.

Remarks

Fetches a byte of data from the FIFO and any associated error conditions. The returned value should be compared to the enumerated type of apUART_eErrorFlags if errors to be detected without interrupts.

6.2.36 apUART_Read_N

Read N bytes of data from UART. Used for reads up to the FIFO size, larger reads should use apUART_RECEIVE.

Declaration

```
PUBLIC UWORD32 apUART_Read_N(apOS_UART_oId Id, UBYTE8* pBuffer, UWORD32 Length)
```

Parameters

Id	UART Id.
pBuffer	pointer to the receiving data buffer.

Return Value

- Number of bytes read.
- apUART_IN_USE if the UART is already receiving.

Remarks

Fetches N bytes of data from the FIFO, where N is less than or equal to FIFO size.

6.2.37 apUART_Receive

Begin receiving data from the UART under interrupt control.

Declaration

```
PUBLIC apError apUART_Receive(apOS_UART_oId Id,
                              void *pBuffer,
                              UWORD32 Size,
                              apUART_rCallback rCallback)
```

Parameters

Id	UART Id.
pBuffer	pointer to the data buffer that will store the received data.
Size	the size of data to receive in bytes.
rCallback	notification function to call when apUART_Receive has finished.

Return Value

- apERR_NONE if successful in setting up receive actions.
- apUART_IN_USE if the UART is already receiving data.
- apUART_NO_BUFFER if pBuffer is aNULL.
- apUART_SIZE_ZERO if the read size is 0.

Remarks

Starts off data fetch and sets up interrupts to keep FIFOs from getting full. A Timer should be set before calling this function with a call to apUART_TerminateReceive and rCallback to ensure the UART doesn't wait for data forever, with rCallback passed apERR_NONE.

6.2.38 apUART_ReceiveStatusGet

Returns the number of remaining bytes that can be read before the pBuffer is full. Primarily designed to be used within the `apUART_rCallback` routine to determine how much of the receive buffer has been filled.

Declaration

```
PUBLIC UWORD32 apUART_ReceiveStatusGet(apOS_UART_oId Id)
```

Parameters

Id	UART Id.

Return Value

Free space remaining in pBuffer.

Remarks

Returns value of static counter.

6.2.39 apUART_RxDisable

Disables the UART's receive ability. (n/a for PL010 and Integrator variants)

Declaration

```
PUBLIC apError apUART_RxDisable(apOS_UART_oId Id)
```

Parameters

Id selects the UART to be referenced.

Return Value

- apERR_NONE if successful.
- apERR_UNSUPPORTED if unsupported by hardware (PL010).

Remarks

Clears the Receive Enable bit in the control register. Receipt of the current byte will complete before the receive section is actually disabled.

6.2.40 apUART_RxEnable

Enables the UART's receive ability. (n/a for PL010 and Integrator variants)

Declaration

```
PUBLIC apError apUART_RxEnable(apOS_UART_oId Id)
```

Parameters

Id selects the UART to be referenced.

Return Value

- apERR_NONE if successful.
- apERR_UNSUPPORTED if unsupported by hardware (PL010).

Remarks

Sets the Receive Enable bit in the control register.

6.2.41 apUART_RxIntDisable

Disables the UART's receive and timeout interrupts.

Declaration

```
PUBLIC void apUART_RxIntDisable(apOS_UART_oId Id)
```

Parameters

Id UART Id.

Remarks

Clears bits in the interrupt mask register.

6.2.42 apUART_RxIntEnable

Enables the UART's receive and timeout interrupts.

Declaration

```
PUBLIC void apUART_RxIntEnable(apOS_UART_oId Id)
```

Parameters

Id	UART Id.

Remarks

Sets bits in the interrupt mask register.

6.2.43 apUART_TerminateReceive

Terminates current receive action. Used in conjunction with apUART_Receive.

Declaration

```
PUBLIC void apUART_TerminateReceive(apOS_UART_oId Id)
```

Parameters

Id	UART Id.

Remarks

Stops further data being read into pBuffer from the Uart. If a Receive/Timeout interrupt occurs after a call to this function the registered rListener routine will be informed in lieu of the rCallback. The registered rCallback routine is also removed and the remaining read buffer size reset to 0.

6.2.44 apUART_TerminateTransmit

Terminates current transmit action. Used in conjunction with apUART_Transmit.

Declaration

```
PUBLIC void apUART_TerminateTransmit(apOS_UART_oId Id)
```

Parameters

Id	UART Id.

Remarks

Disables the transmit interrupt. Stops any additional data being written to the Tx FIFO. FIFO continues to empty if applicable.

6.2.45 apUART_Transmit

Begin sending data to the UART for transmission under interrupt control.

Declaration

```
PUBLIC apError apUART_Transmit(apOS_UART_oId Id,
                               void *pBuffer,
                               UWORD32 Size,
                               apUART_rCallback rCallback)
```

Parameters

Id	UART Id.
pBuffer	pointer to the buffer with the data to be transmitted
Size	the size of data to transmit in bytes
rCallback	pointer to a notification function that apUART_Transmit has finished

Return Value

- apERR_NONE if successful in setting up transmit actions.

- apUART_IN_USE if the UART is already transmitting data.
- apUART_NO_BUFFER if pBuffer is aNULL.
- apUART_SIZE_ZERO if the transmit size is 0.

Remarks

Starts off data transfer and sets up the Transmit interrupt to keep the Tx FIFO fed. The Transmit interrupt is disabled when the transfer has completed.

6.2.46 apUART_TransmitStatusGet

Returns the number of bytes that remain to be written to the FIFO. Used in conjunction with apUART_Transmit().

Declaration

```
PUBLIC UWORD32 apUART_TransmitStatusGet(apOS_UART_oId Id)
```

Parameters

Id UART Id.

Return Value

Number of bytes still to be written to Tx FIFO in current transfer.

Remarks

Returns value of static counter.

6.2.47 apUART_TxDisable

Disables the UART's transmit ability. (n/a for PL010 and Integrator variants)

Declaration

```
PUBLIC apError apUART_TxDisable(apOS_UART_oId Id)
```

Parameters

Id selects the UART to be referenced.

Return Value

- apERR_NONE if successful.
- apERR_UNSUPPORTED if unsupported by hardware (PL010).

Remarks

Clears the Transmit Enable bit in the control register. Transmission of the current byte will complete before transmit is disabled.

6.2.48 apUART_TxEnable

Enables UART transmit ability. (n/a for PL010 and Integrator variants)

Declaration

```
PUBLIC apError apUART_TxEnable(apOS_UART_oId Id)
```

Parameters

Id selects the UART to be referenced.

Return Value

- apERR_NONE if successful.
- apERR_UNSUPPORTED if unsupported by hardware (PL010).

Remarks

Sets the Transmit Enable bit in the control register. .

6.2.49 apUART_TxFIFOEmpty

Returns a boolean indicating if the Transmit FIFO is empty. Primarily for use in conjunction with apUART_Write_N when writing FIFO-size blocks of data.

Declaration

```
PUBLIC BOOL apUART_TxFIFOEmpty(apOS_UART_oId Id)
```

Parameters

Id selects the UART to be referenced.

Return Value

- TRUE if the Transmit FIFO is empty.
- FALSE if the Transmit FIFO is not empty.

Remarks

Checks the flag register to see if the Transmit FIFO is empty.

6.2.50 apUART_TxIntDisable

Disables the UART's transmit interrupt. Used to pause the current apUART_Transmit action.

Declaration

```
PUBLIC void apUART_TxIntDisable(apOS_UART_oId Id)
```

Parameters

Id UART Id.

Remarks

Clears a bit in the interrupt mask register.

6.2.51 apUART_TxIntEnable

Enables the UART's transmit interrupt. Used to restart the apUART_Transmit action.

Declaration

```
PUBLIC void apUART_TxIntEnable(apOS_UART_oId Id)
```

Parameters

Id UART Id.

Remarks

Sets a bit in the interrupt mask register.

6.2.52 apUART_Write

Send a byte of data to UART.

Declaration

```
PUBLIC apError apUART_Write(apOS_UART_oId Id, UWORD32 Byte_to_send)
```

Parameters

Id	UART Id.
Byte_to_send	data value to send.

Return Value

- apERR_NONE for success.
- apUART_FIFO_FULL if there is no space free in the transmit FIFO.
- apUART_IN_USE if the UART is already in use.

Remarks

Places a byte of data on the Tx FIFO.

6.2.53 apUART_Write_N

Send N bytes of data to the UART's Tx FIFO. Used for writes up to the Tx FIFO size, larger writes should use apUART_TRANSMIT.

Declaration

```
PUBLIC UWORD32 apUART_Write_N(apOS_UART_oId Id,
                              UBYTE8* pBuffer,
                              UWORD32 Length)
```

Parameters

Id	UART Id.
pBuffer	pointer to the data to send.
Length	the number of bytes of data to transmit.

Return Value

- Number of bytes written.
- apUART_IN_USE - if the UART is already transmitting.

Remarks

Places N bytes of data on FIFO, where N <= FIFO size.

6.2.54 apUART_IntHandler

Standard interrupt handler for UART module.

Declaration

```
PUBLIC void apUART_IntHandler(CONST apOS_INT_oInterruptSource oInterruptId,
                              UWORD32 DeviceId)
```

Parameters

oInterruptId	The ID of the interrupt
DeviceId	Identifier for the instance of the driver

Return Value

none

Remarks

This function should only be called by an interrupt dispatcher.

6.2.55 apUART_RawISR

This is the raw interrupt handler for the module, to be called directly from the interrupt vector.

Declaration

```
IRQ void apUART_RawISR(void)
```

Parameters

none

Return Value

none

Remarks

This routine should only be executed as a branch from the IRQ vector.

6.2.56 apUART_StateSizeGet

Retrieves the size required for storage of the driver state data.

Declaration

```
PUBLIC UWORD32 apUART_StateSizeGet(void)
```

Parameters

none

Return Value

size (in bytes) required

Remarks

This function is required if apOS_NO_STATIC_STATE is defined as TRUE.